UNIVERSITY OF ASIA PACIFIC

Department of Computer Science and Engineering

CSE 402: Operating System Lab

LAB REPORT

Report Number: 5

Report Title: SJF (Preemptive/SRTF) CPU Scheduling

Submitted by:
Name: Md. Muhaimin Al Baki Raiyan
Student ID: 23101163
Semester: 4th Year, 1st Semester
Section: C-2

Submitted To:
Atia Rahman Orthi
Lecturer,
Department of Computer Science and Engineering

Date of Submission: 29.08.2026

Objective

The objective of this lab exercise is to implement the Preemptive Shortest Job First (SJF) CPU scheduling algorithm, also known as Shortest Remaining Time First (SRTF), using a Time Quantum of 2 units. Specifically, the lab aims to:

- Implement Preemptive SJF scheduling for a given set of processes with different Arrival Times (AT) and Burst Times (BT).
- Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process.
- Compute the Average Turnaround Time and Average Waiting Time for the resulting schedule.
- Compare the performance of Preemptive SJF against Non-Preemptive SJF for the same process set.

SJF (Preemptive/SRTF)

Preemptive Shortest Job First (SJF), commonly known as Shortest Remaining Time First (SRTF), is a CPU scheduling algorithm that, at every point in time, executes the process with the smallest remaining burst time among all processes that have already arrived. Unlike the non-preemptive version of SJF, a running process is not guaranteed to run to completion: if a new process arrives with a remaining burst time smaller than that of the currently executing process, the CPU is immediately taken away from the running process and given to the new arrival.

This preemptive behavior allows short processes to be completed quickly and generally results in a lower average waiting time and turnaround time compared to non-preemptive scheduling. However, it comes at the cost of additional context switching, since the CPU may need to repeatedly interrupt and resume processes as shorter jobs arrive. In practice, SJF/SRTF is often approximated using a small time quantum (as in this lab, TQ = 2) to periodically re-check the ready queue for preemption opportunities.

Algorithm

- Sort all the given processes in ascending order of their Arrival Time (AT).
- Initialize the remaining burst time of every process to its given Burst Time (BT), and set the current time to 0.
- At the current time, form the ready queue consisting of all processes that have arrived (AT ≤ current time) and have not yet finished execution.
- If the ready queue is empty, advance the current time directly to the arrival time of the next process.
- From the ready queue, select the process with the smallest remaining burst time (ties are broken by earlier arrival time).
- Execute the selected process for one Time Quantum (2 units), or until it either finishes or a new process arrives — whichever happens first.
- Reduce the selected process's remaining burst time by the amount it was executed, and advance the current time accordingly.
- If a process's remaining burst time becomes 0, record its Completion Time (CT), then compute its Turnaround Time (TAT = CT − AT) and Waiting Time (WT = TAT − BT).
- Repeat the above steps until all processes have completed execution.

- Finally, compute the Average Turnaround Time and Average Waiting Time across all processes.

1. Problem:

Given the following processes with their **Arrival Time (AT)** and **Burst Time (BT)**:

Process	Arrival Time (AT)	Burst Time (BT)
P1	4	2
P2	2	2
P3	1	3
P4	0	6
P5	3	1

Implement the **Preemptive Shortest Job First (SJF)** CPU Scheduling Algorithm using a **Time Quantum (TQ) of 2 units**.

Calculate the following for each process:

- **Completion Time (CT)**
- **Turnaround Time (TAT)**
- **Waiting Time (WT)**

Also, calculate:

- **Average Turnaround Time (Average TAT)**
- **Average Waiting Time (Average WT)**

Finally, display the results for all processes.

2. Source Code:

```
processes = [
    {'pid': 'P1', 'at': 4, 'bt': 2},
    {'pid': 'P2', 'at': 2, 'bt': 2},
    {'pid': 'P3', 'at': 1, 'bt': 3},
    {'pid': 'P4', 'at': 0, 'bt': 6},
    {'pid': 'P5', 'at': 3, 'bt': 1},

]

TQ = 2


def sjf_preemptive_tq(processes, tq):

    plist = sorted(processes, key=lambda p: p['at'])
    remaining_time = {p['pid']: p['bt'] for p in plist}
    at_of = {p['pid']: p['at'] for p in plist}
    bt_of = {p['pid']: p['bt'] for p in plist}

    gantt = []
    completed = {}
    time = 0
    n = len(plist)
    finished = 0
    max_time_guess = sum(p['bt'] for p in plist) + max(p['at'] for p in plist)
```

```
    while finished < n and time <= max_time_guess:
        ready = [p for p in plist if p['at'] <= time and
remaining_time[p['pid']] > 0]

        if not ready:
            future = [p['at'] for p in plist if remaining_time[p['pid']] > 0]
            time = min(future)
            continue


        current = min(ready, key=lambda p: (remaining_time[p['pid']],
p['at']))
        pid = current['pid']


        run_for = min(tq, remaining_time[pid])


        future_arrivals = [p['at'] for p in plist if p['at'] > time and
p['at'] < time + run_for]
        if future_arrivals:
            run_for = min(future_arrivals) - time

        start = time
        end = time + run_for

        if gantt and gantt[-1][0] == pid and gantt[-1][2] == start:
            gantt[-1] = (pid, gantt[-1][1], end)
        else:
            gantt.append((pid, start, end))

        remaining_time[pid] -= run_for
        time = end

        if remaining_time[pid] == 0:
            ct = time
            tat = ct - at_of[pid]
            wt = tat - bt_of[pid]
            completed[pid] = {'pid': pid, 'at': at_of[pid], 'bt': bt_of[pid],
'ct': ct, 'tat': tat, 'wt': wt}
            finished += 1

    return [completed[p['pid']] for p in plist], gantt


def get_averages(completed):
    total_tat = sum(p['tat'] for p in completed)
    total_wt = sum(p['wt'] for p in completed)
    n = len(completed)
    return total_tat / n, total_wt / n


def print_results(name, completed, gantt):
    print(f"{name} ")
    print("Gantt Chart:")
    print(" | ".join(f"{pid}({s}-{e})" for pid, s, e in gantt))
    print()
    print(f"{'PID':<5}{'AT':<5}{'BT':<5}{'CT':<5}{'TAT':<5}{'WT':<5}")
    for p in sorted(completed, key=lambda x: x['pid']):
        print(f"{p['pid']:<5}{p['at']:<5}{p['bt']:<5}{p['ct']:<5}{p['tat']:<5}
{p['wt']:<5}")
    avg_tat, avg_wt = get_averages(completed)
    print(f"\nAverage TAT: {avg_tat:.2f}")
```

```
    print(f"Average WT: {avg_wt:.2f}\n")
    return avg_tat, avg_wt


completed, gantt = sjf_preemptive_tq(processes, TQ)
print_results(f"SJF (Preemptive / SRTF, TQ={TQ})", completed, gantt)
```

3. Output:

```
PS G:\All Notes\Operating System Lab> python sjf_preemptive_tq.py
SJF (Preemptive / SRTF, TQ=2)
Gantt Chart:
P4(0-1) | P3(1-4) | P5(4-5) | P2(5-7) | P1(7-9) | P4(9-14)

PID   AT    BT    CT    TAT   WT
P1    4     2     9     5     3
P2    2     2     7     5     3
P3    1     3     4     3     0
P4    0     6     14    14    8
P5    3     1     5     2     1

Average TAT: 5.80
Average WT: 3.00

PS G:\All Notes\Operating System Lab>
```

4. Discussion:

In this experiment, Preemptive Shortest Job First (SJF) with a Time Quantum of 2 units and Non-Preemptive Shortest Job First (SJF) CPU scheduling techniques were applied to the same five processes. Their performances were compared by analyzing Completion Time (CT), Turnaround Time (TAT), Waiting Time (WT), Average TAT, and Average WT.

The Preemptive SJF approach achieved an Average TAT of 5.80 and an Average WT of 3.00. In this scheduling method, the CPU re-evaluates the ready queue whenever a new process arrives, and switches to whichever ready process currently has the smallest remaining burst time. As a result, short processes such as P5 (BT = 1) and P3 (BT = 3, which repeatedly won tie-breaks due to its earlier arrival) complete quickly with very little or no waiting, while the longest process, P4 (BT = 6), is pushed to the end and accumulates the largest waiting time (WT = 8).

In contrast, the Non-Preemptive SJF method resulted in an Average TAT of 7.40 and an Average WT of 4.60. Here, a process keeps the CPU until its execution is finished, even if a shorter process arrives during that time — for example, once P4 starts at t = 0, it occupies the CPU for its full burst of 6 units even though P3, P2, and P5 all arrive and become ready before it finishes. This causes shorter processes to remain in the ready queue for a longer period than under the preemptive approach.

From the obtained results, it is clear that Preemptive SJF achieved a lower average turnaround time and waiting time than Non-Preemptive SJF for this process set (5.80 vs 7.40 for TAT, and 3.00 vs 4.60 for WT). Thus, Preemptive SJF demonstrates better

scheduling efficiency for the selected process set, at the cost of additional context switching between processes.

It is worth noting that the maximum number of students in the class who solved this same problem obtained a slightly higher Average TAT of 6.00 and Average WT of 3.20 for the identical Preemptive SJF (TQ = 2) problem set, compared to the Average TAT of 5.80 and Average WT of 3.00 obtained here. On closer comparison of the two implementations, **this gap does not come from a difference in the scheduling policy itself, but from how often each program re-checks the ready queue for a possible preemption**. Most students' implementation only re-evaluates which process has the shortest remaining burst time at fixed quantum boundaries (i.e., every 2 time units, or when a process finishes early); a newly arrived process must therefore wait until the next boundary before it can preempt the CPU. For example, P4 starts running at t = 0 and keeps the CPU for the full quantum until t = 2, even though P3 (with a shorter remaining burst time) arrives at t = 1; because that early opportunity to preempt is missed, P3 is repeatedly pushed back and does not finish until t = 10, giving it a Waiting Time of 6. In this implementation, the ready queue is instead re-checked continuously, and a running process is cut short the instant a process with a smaller remaining burst time arrives, even if that happens mid-quantum. As a result, P4's first run is preempted almost immediately, at t = 1, letting P3 start right away and finish by t = 4 with a Waiting Time of only 0. This closer, event-driven approximation of true preemptive SJF is what pulls the Average TAT and Average WT slightly below the values obtained from the quantum-boundary-only version, even though both programs are built on the same scheduling algorithm and the same input data.

To verify which of the two results is the more accurate implementation of Preemptive SJF, a reference version of the algorithm was also implemented in continuous time (i.e., re-checking the ready queue at every unit of time rather than only at fixed 2-unit boundaries), since true preemptive SJF, also known as Shortest Remaining Time First (SRTF), has no inherent notion of a time quantum and is defined to preempt the CPU the instant a shorter job becomes ready. This reference implementation produced an Average TAT of 5.80 and an Average WT of 3.00, with the same Gantt chart sequence of P4(0-1), P3(1-4), P5(4-5), P2(5-7), P1(7-9), P4(9-14) obtained here, confirming that this implementation's event-driven preemption matches the theoretically correct SRTF schedule exactly. The quantum-boundary version used by most of the class, by only re-evaluating the ready queue every 2 time units, **misses the valid preemption opportunity at t = 1** and therefore produces a schedule that deviates from true preemptive SJF, yielding the slightly higher Average TAT of 6.00 and Average WT of 3.20.

5. Conclusion:

The experiment was carried out successfully to implement and compare Preemptive and Non-Preemptive SJF scheduling algorithms using a Time Quantum of 2 units for the preemptive variant. Based on the calculated results, Preemptive SJF provided an Average TAT of 5.80 and an Average WT of 3.00, whereas Non-Preemptive SJF produced an Average TAT of 7.40 and an Average WT of 4.60.

These results indicate that Preemptive SJF is more efficient for this particular set of processes because it reduces both turnaround time and waiting time by allowing shorter,

later-arriving processes to interrupt a longer one that is already running. However, Non-Preemptive SJF has the advantage of being simpler to implement and free of context-switching overhead, since a process is allowed to run to completion without interruption once it begins execution.